

Universidad Abierta para Adultos (UAPA)

Escuela de Ingeniería y Tecnología

Carrera de Ingeniería de Software

INFORME TÉCNICO DE INVESTIGACIÓN APLICADA

Sistema de Gestión Clínica — ClinicaSystem

Asignatura: Ingeniería de Software I

Sustentante: Carlos Daniel Flores

Matrícula: 100081538

Fecha: 10/1/2026

INTRODUCCIÓN

El presente informe técnico documenta el proceso de análisis, diseño y desarrollo del sistema ClinicaSystem, una aplicación de escritorio para la gestión integral de una clínica médica, desarrollada como proyecto integrador de la asignatura Ingeniería de Software I. El propósito de este documento es presentar de forma coherente e integrada los distintos productos generados a lo largo del curso análisis de requisitos, modelado UML, diseño arquitectónico, gestión de avance y control técnico del proyecto, demostrando la aplicación práctica de los fundamentos de la ingeniería de software sobre un sistema real y funcional.

El desarrollo de ClinicaSystem partió de la necesidad de digitalizar y ordenar los procesos administrativos y clínicos de un centro de salud pequeño o mediano: registro de pacientes, gestión de médicos, programación de citas, historial médico y control de acceso por roles de usuario. A lo largo del curso, cada entrega semanal abordó una etapa distinta del ciclo de vida del software, desde el levantamiento de requisitos hasta la definición de la arquitectura y la gestión del proyecto mediante herramientas colaborativas como GitHub y GitHub Projects.

Este informe integra dicho trabajo en un solo documento técnico, evidenciando no solo los artefactos producidos (diagramas, tableros, repositorio), sino también el razonamiento y las decisiones técnicas que sustentan cada uno, en relación directa con la interfaz y el funcionamiento real del sistema.

DESARROLLO DEL INFORME

1. Presentación del sistema

ClinicaSystem es una aplicación de escritorio desarrollada en C# con Windows Forms (.NET), respaldada por una base de datos SQL Server (ClinicaDB), diseñada para automatizar la gestión operativa de una clínica médica.

Problema que atiende

Centros de salud pequeños o medianos que gestionan de forma manual o dispersa el registro de pacientes, la agenda de médicos y el historial clínico, con el consiguiente riesgo de citas duplicadas, pérdida de información y acceso no controlado a los datos.

Usuarios principales, objetivo y contexto de uso

El sistema está dirigido a tres perfiles de usuario: Administrador, Recepcionista y Médico, cada uno con acceso restringido a los módulos correspondientes. Su objetivo es centralizar y controlar los procesos de registro de pacientes, gestión de médicos, programación de citas e historial médico dentro de un único sistema de escritorio, de uso diario en el punto de atención de la clínica.

Alcance del proyecto

- ◆ Gestión completa (CRUD) de Pacientes, Médicos, Citas, Historial médico y Usuarios.
- ◆ Control de acceso basado en roles, con tres perfiles diferenciados Administrador, Recepcionista y Médico, cada uno con acceso restringido a los módulos correspondientes a través del menú principal (FrmMenuPrincipal).
- ◆ Seguridad de acceso: las contraseñas se almacenan mediante hash PBKDF2 con salt (con migración transparente desde un esquema anterior basado en SHA-256), y el sistema bloquea una cuenta tras 5 intentos fallidos de inicio de sesión.
- ◆ Acceso a datos centralizado mediante la clase DbHelper, que unifica la ejecución de consultas y el manejo de errores SQL (claves foráneas, duplicados, timeouts, fallos de conexión) traducidos a mensajes en español mediante DatosException.
- ◆ Interfaz con tema oscuro (#1E3A5F / #16284B), aplicado de manera consistente en todos los formularios del sistema.

El código fuente se encuentra públicamente disponible en el repositorio de GitHub: github.com/Carlosrose07/ClinicaSystem.

Objetivos del proyecto

Objetivo general: desarrollar un sistema de escritorio que digitalice y controle los procesos administrativos y clínicos esenciales de una clínica médica pequeña o mediana, aplicando principios de ingeniería de software en cada etapa de su construcción.

Objetivos específicos:

1. Especificar los requisitos funcionales y no funcionales del sistema, con criterios de aceptación verificables.
2. Modelar el sistema mediante diagramas UML que representen su alcance funcional y su flujo principal de trabajo.
3. Diseñar una arquitectura en capas que separe responsabilidades entre presentación, lógica de negocio, acceso a datos y base de datos.
4. Implementar un mecanismo de autenticación y control de acceso por rol, con almacenamiento seguro de credenciales.
5. Gestionar el avance del proyecto mediante un tablero Kanban con tareas, prioridades, fechas y riesgos.

6. Mantener control técnico del proyecto mediante repositorio Git, documentación (README) y registro de cambios.

Ficha técnica:

PROPIEDAD	DETALLE
Nombre del sistema	ClinicaSystem
Lenguaje	C#
Framework / UI	.NET Windows Forms
Base de datos	SQL Server Express (ClinicaDB)
Patrón de diseño	Arquitectura en capas + Repository
Conector de datos	ADO.NET / SqlConnection
Plataforma	Windows (aplicación de escritorio)
IDE	Visual Studio 2019

Operaciones CRUD principales:

OPERACIÓN	ACCIÓN SQL	MÉTODO EN REPOSITORIO	DESCRIPCIÓN
Crear	INSERT	EjecutarNonQuery	Registra un nuevo paciente, médico, cita o usuario.
Leer	SELECT	EjecutarConsulta<T> / EjecutarConsultaUnica<T>	Obtiene el listado o un registro específico.
Actualizar	UPDATE	EjecutarNonQuery	Modifica los datos de un registro existente.
Eliminar	DELETE	EjecutarNonQuery	Elimina un registro, validando dependencias.

Stack tecnológico:

TECNOLOGÍA	VERSIÓN	FUNCIÓN EN EL PROYECTO
C# / .NET	.NET Framework 4.7.2	Lenguaje y framework de desarrollo de la aplicación.
Windows Forms		Construcción de la interfaz de usuario de escritorio.
SQL Server	Sql Server Express	Motor de base de datos (ClinicaDB).
GitHub / GitHub Projects		Control de versiones y gestión Kanban del proyecto.
Visual Paradigm Online		Elaboración de los diagramas UML.

Esta arquitectura repositorios especializados por entidad, acceso a datos centralizado y separación de responsabilidades por capas es la base sobre la cual se construyeron el modelado UML y el diseño arquitectónico documentados en las secciones siguientes.

2. Enfoque de desarrollo

Para la gestión del proyecto ClinicaSystem se adoptó un enfoque ágil basado en Kanban, implementado mediante la herramienta GitHub Projects, en lugar de un enfoque tradicional (cascada) o de un marco ágil más prescriptivo como Scrum. Esta decisión se justifica en los siguientes puntos:

◆ **Naturaleza del proyecto y del equipo:** ClinicaSystem fue desarrollado de forma individual y por etapas semanales, con requisitos que se refinaron progresivamente a lo largo del curso. Un enfoque tradicional (cascada), que exige

requisitos completamente cerrados antes de iniciar el diseño y la implementación, no se ajustaba a esta dinámica iterativa.

◆ **Visibilidad continua del estado del trabajo:** Kanban permite visualizar en todo momento qué tareas están en Backlog, en progreso, en revisión o finalizadas, sin necesidad de ceremonias formales de sprint como en Scrum; apropiado para un desarrollador único que necesita organización sin sobrecarga de proceso.

◆ **Flexibilidad ante cambios:** al no trabajar en iteraciones de tiempo fijo (sprints), las tareas pueden reordenarse por prioridad según necesidades emergentes, sin romper un plan cerrado.

◆ **Integración natural con GitHub:** al usar GitHub Projects junto con Issues vinculados a cada tarea, el tablero Kanban se integra directamente con el control de versiones y el historial de commits, dando trazabilidad entre la planificación y el código real.

◆ **Evidencia de aplicación real:** el tablero de GitHub Projects construido en la Semana VI (10 tareas clasificadas por tipo, prioridad, responsable, fecha estimada y estado) demuestra la aplicación consistente de este enfoque a lo largo del curso (Anderson, 2010).

En conjunto, este enfoque ágil en la gestión del flujo de trabajo (Kanban), pero apoyado en documentación técnica formal (UML, arquitectura, requisitos) permitió balancear la flexibilidad necesaria para un proyecto académico individual con el rigor documental exigido por la asignatura.

3. Especificación de requisitos

Los requisitos de ClinicaSystem fueron levantados durante la Semana III del curso mediante observación directa del flujo de trabajo de una clínica y entrevistas informales sobre necesidades del personal administrativo y médico. A partir de 5 hallazgos principales se derivaron 5 requisitos funcionales (RF) y 5 requisitos no funcionales (RNF), cada uno acompañado de un criterio de aceptación verificable, que sirvió posteriormente como base para el modelado UML (Semana IV) y las pruebas de validación durante el desarrollo.

Requisitos funcionales:

CÓDIGO	DESCRIPCIÓN	PRIORIDAD	CRITERIO DE ACEPTACIÓN
RF-01	Registrar, editar, eliminar y listar pacientes con validación de campos obligatorios.	Alta	El sistema rechaza el registro si algún campo obligatorio está vacío y guarda correctamente los datos válidos.
RF-02	Registrar citas médicas validando que el doctor no tenga otra cita en el mismo horario.	Alta	El sistema no permite guardar una cita si el doctor ya tiene otra registrada en el mismo rango de horario.
RF-03	Autenticar usuarios mediante usuario y contraseña, validados contra el hash almacenado.	Alta	El sistema deniega el acceso si las credenciales no coinciden con el hash PBKDF2 almacenado.
RF-04	Mostrar opciones de menú según el rol del usuario autenticado (Admin, Recepcionista, Médico).	Alta	Cada rol visualiza únicamente las opciones de menú permitidas para su perfil.
RF-05	Mostrar al médico únicamente las citas del día correspondientes a su usuario.	Media	El médico visualiza solo sus propias citas del día actual al iniciar sesión.

Requisitos no funcionales

CÓDIGO	DESCRIPCIÓN	PRIORIDAD	CRITERIO DE ACEPTACIÓN
RNF-01	Almacenar las contraseñas mediante PBKDF2 con salt único por usuario.	Alta	Ninguna contraseña se almacena en texto plano; cada usuario tiene un salt distinto.
RNF-02	Bloquear la cuenta de forma temporal tras 5 intentos fallidos de inicio de sesión.	Alta	Tras 5 intentos fallidos consecutivos, la cuenta se bloquea automáticamente durante 15 minutos.
RNF-03	Ofrecer una interfaz comprensible para personal sin experiencia técnica avanzada.	Media	Un usuario nuevo completa el registro de un paciente en menos de 3 minutos sin capacitación previa.
RNF-04	Responder las consultas de listados (pacientes, citas) en un tiempo aceptable.	Media	Las listas de pacientes o citas cargan en 2 segundos o menos con hasta 1000 registros.
RNF-05	Mostrar mensajes de error claros ante fallos de conexión a la base de datos, sin exponer detalles técnicos.	Media	Ante un fallo de conexión SQL, el sistema muestra un mensaje comprensible mediante DatosException, sin detalles internos.

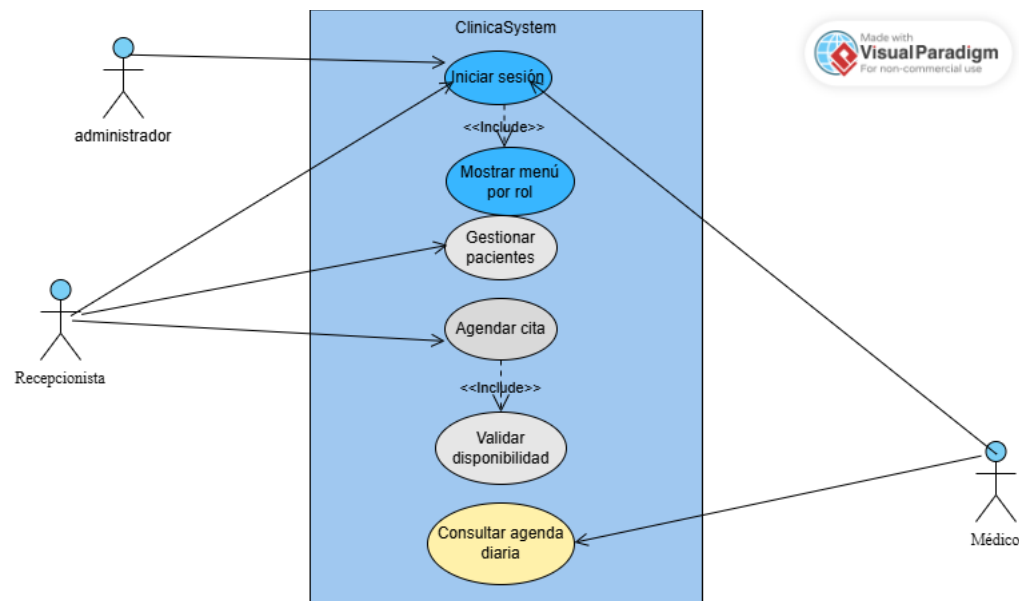
Cabe destacar que estos criterios de aceptación no quedaron únicamente en el papel: varios de ellos se reflejan directamente en la implementación actual del sistema. Por ejemplo, el criterio de RNF-01 se verifica en el uso de PBKDF2 con salt en el módulo de autenticación (Kaliski, 2000); el de RNF-02, en el bloqueo tras 5 intentos fallidos; y el de RNF-05, en el manejo centralizado de errores mediante DbHelper y DatosException, que traduce los códigos de error SQL (547, 2627/2601, -2, 53/4060/18456) a mensajes comprensibles para el usuario final.

4. Modelado técnico

A partir de los requisitos funcionales RF-01 a RF-05, se desarrolló un conjunto de diagramas UML en Visual Paradigm Online, con el objetivo de representar el sistema desde distintas perspectivas complementarias antes de su implementación completa (Fowler, 2003):

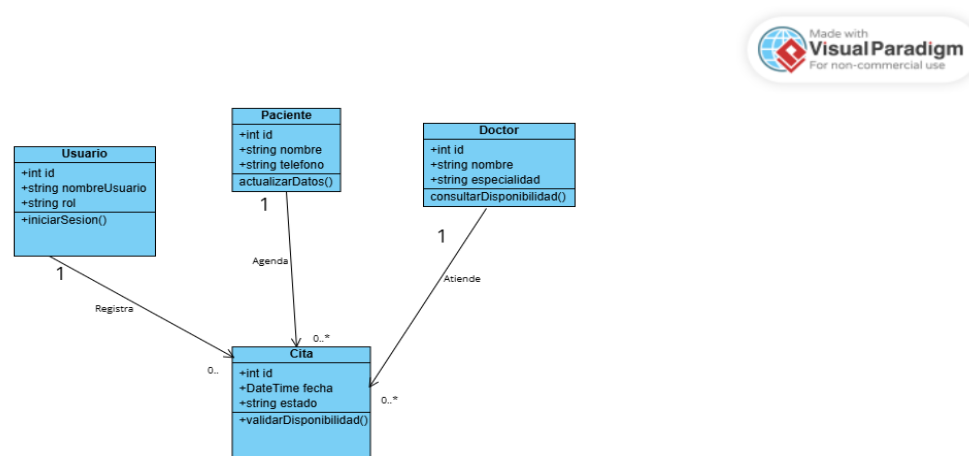
◆ **Diagrama de casos de uso:** modela la interacción de tres actores (Administrador, Recepcionista, Médico) con seis casos de uso principales, incluyendo relaciones incluye (por ejemplo, "Iniciar sesión" incluye "Mostrar menú por rol", y "Agendar cita" incluye "Validar disponibilidad"). Aporta una visión funcional de alto nivel: qué puede hacer cada tipo de usuario en el sistema, sin entrar en detalles de implementación.

Figura 1. Diagrama de casos de uso de ClinicaSystem.



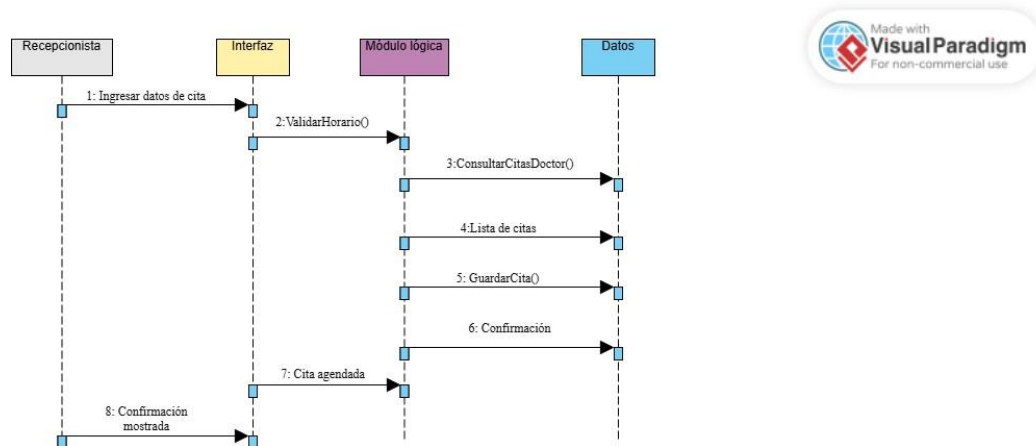
◆ **Diagrama de clases:** representa las entidades principales del dominio (Usuario, Paciente, Doctor, Cita) con sus atributos, métodos y relaciones. Aporta la estructura estática del sistema y sirvió como referencia directa para el diseño de las clases de entidad y los repositorios implementados en C#.

Figura 2. Diagrama de clases de ClinicaSystem.



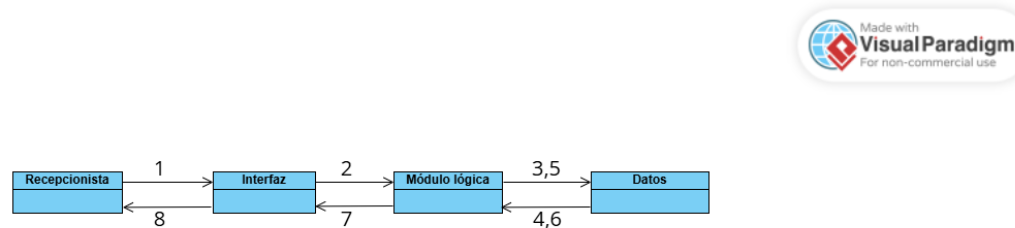
◆ **Diagrama de secuencia:** documenta el flujo temporal del caso de uso "Agendar cita" (RF-02) en su camino feliz, con 8 mensajes intercambiados entre Recepcionista, Interfaz, Módulo de lógica y Datos. Aporta claridad sobre el orden de las operaciones y qué componente es responsable de cada paso, incluyendo la validación de disponibilidad antes de confirmar la cita.

Figura 3. Diagrama de secuencia Agendar cita.



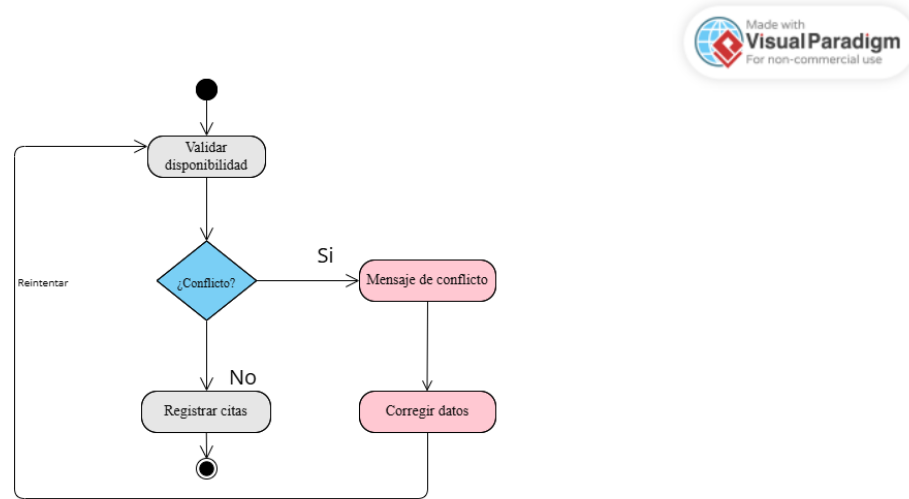
◆ **Diagrama de colaboración:** representa la misma interacción que el de secuencia, pero enfatizando las relaciones estructurales entre los objetos participantes en lugar del orden temporal. Aporta una vista complementaria útil para verificar que no se omitió ninguna dependencia entre componentes.

Figura 4. Diagrama de colaboración Agendar cita.



◆ **Diagrama de actividades:** modela el flujo completo de "Agendar cita", incluyendo el nodo de decisión de disponibilidad y el camino alternativo en caso de conflicto de horario, con retorno (loop-back) a la validación antes de reintentar. Aporta la lógica de control y manejo de excepciones del proceso, directamente relacionada con el criterio de aceptación de RF-02.

Figura 5. Diagrama de actividades Agendar cita.



La validación técnica de estos diagramas confirmó su coherencia entre sí: las entidades del diagrama de clases coinciden con los actores y objetos del diagrama de secuencia/colaboración, y el flujo del diagrama de actividades refleja fielmente el camino alternativo de conflicto de horario exigido por RF-02.

Diagrama de flujo CRUD

De forma complementaria a los diagramas UML, el flujo de una operación CRUD típica del sistema por ejemplo, guardar un paciente sigue seis pasos consecutivos entre la interfaz y la base de datos:

//flujo-crud		
USUARIO	(INICIO)	Interactúa con un formulario
EVENTO	(TRIGGER)	Clic en "Guardar" dispara el evento
REPOSITORIO	(LOGICA)	PacienteRepository valida los datos
SQL QUERY	(QUERY)	DbHelper ejecuta la consulta parametrizada
SQL SERVER	(PERSISTENCIA)	ClinicaDB inserta o actualiza el registro
RESULTADO	(FEEDBACK)	La interfaz confirma o muestra el error

Figura 6. Flujo general de una operación CRUD en ClinicaSystem.

5. Diseño arquitectónico

ClinicaSystem se diseñó siguiendo una arquitectura en capas (layered architecture) (Bass et al., 2013), compuesta por:

Capa 1 — Presentación

Formularios Windows Forms (FrmMenuPrincipal y demás pantallas), responsables de la captura de interacción del usuario.

Capa 2 Lógica de negocio

Validaciones de dominio por ejemplo, verificación de disponibilidad de horario antes de agendar una cita.

Capa 3 Acceso a datos (Repositorios)

Repositorios especializados por entidad (UsuarioRepository, PacienteRepository, MedicoRepository, etc.), apoyados en la clase centralizada DbHelper.

Capa 4 Base de datos

SQL Server (ClinicaDB).

A estas se suma Autenticación como un servicio transversal, conectado únicamente a las capas de Presentación y Lógica de negocio, ya que el control de acceso por roles debe aplicarse antes de permitir el uso de cualquier funcionalidad, pero no depende directamente del acceso a datos crudo.

Tabla de componentes

COMPONENTE	RESPONSABILIDAD	RELACIONES
Presentación	Formularios y captura de interacción del usuario	Se comunica con Lógica de negocio y Autenticación
Lógica de negocio	Reglas de dominio y validaciones (ej. conflicto de citas)	Recibe datos de Presentación, consulta Acceso a datos
Acceso a datos	Ejecución de consultas SQL vía DbHelper y repositorios	Recibe solicitudes de Lógica de negocio, consulta Base de datos
Base de datos	Persistencia de la información (ClinicaDB)	Accedida únicamente por Acceso a datos
Autenticación	Validación de credenciales y control de acceso por rol	Conectada a Presentación y Lógica de negocio

Justificación de la decisión arquitectónica: se eligió una arquitectura en capas porque coincide naturalmente con la organización ya existente del código base (formularios WinForms, clases Repository por entidad, y DbHelper como capa de acceso a datos), evitando una reestructuración innecesaria del proyecto. Este estilo aplica el principio de separación de responsabilidades (Separation of Concerns), facilitando el mantenimiento: un cambio en la lógica de validación de citas no requiere modificar el acceso a datos ni la interfaz. Como riesgo o limitación, una arquitectura en capas estrictamente secuencial puede introducir cierta latencia en operaciones que atraviesan varias capas para tareas simples, aunque este costo se consideró aceptable dado el volumen de datos esperado para una clínica pequeña o mediana (coherente con RNF-04).

Este diseño quedó registrado como la tarjeta "Definición de arquitectura inicial" (#9) en el tablero de GitHub Projects, con la justificación técnica y el enlace al diagrama en su descripción.

6. Gestión del proyecto

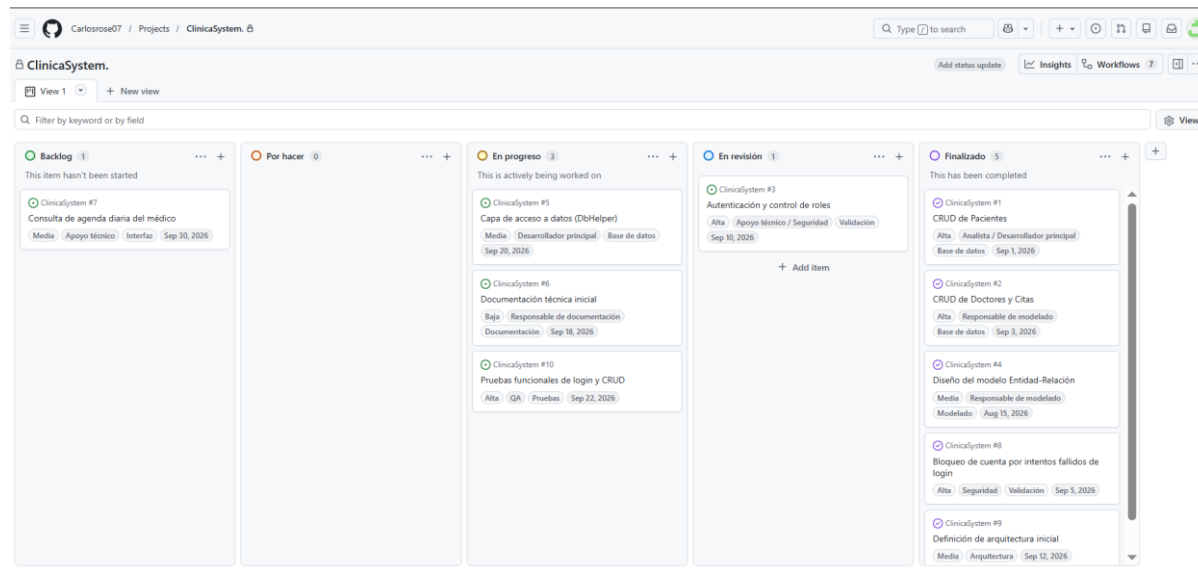
El avance de ClinicaSystem se gestionó mediante un tablero Kanban en GitHub Projects, vinculado directamente al repositorio Carlosrose07/ClinicaSystem. El tablero organiza el trabajo en cinco estados (Backlog, Por hacer, En progreso, En revisión, Finalizado) y clasifica cada tarea con los campos Prioridad, Responsable, Tipo y Fecha estimada.

Tareas registradas

#	TAREA	ESTADO	PRIORIDAD	RESPONSABLE	TIPO	FECHA
1	CRUD de Pacientes	Finalizado	Alta	Desarrollador principal	Base de datos	1 sep 2026
2	CRUD de Doctores y Citas	Finalizado	Alta	Responsable de modelado	Base de datos	3 sep 2026
3	Autenticación y control de roles	En revisión	Alta	Seguridad	Validación	10 sep 2026
4	Diseño del modelo Entidad-Relación	Finalizado	Media	Responsable de modelado	Modelado	15 ago 2026
5	Capa de acceso a datos (DbHelper)	En progreso	Media	Desarrollador principal	Base de datos	20 sep 2026
6	Documentación técnica inicial	En progreso	Baja	Responsable de documentación	Documentación	18 sep 2026
7	Consulta de agenda diaria del médico	Backlog	Media	Apoyo técnico	Interfaz	30 sep 2026
8	Bloqueo de cuenta por intentos fallidos de login	Finalizado	Alta	Seguridad	Validación	5 sep 2026
9	Definición de arquitectura inicial	Finalizado	Media	—	Arquitectura	12 sep 2026
10	Pruebas funcionales de login y CRUD	En progreso	Alta	QA	Pruebas	22 sep 2026

Distribución final: 1 en Backlog, 0 en Por hacer, 3 En progreso, 1 En revisión, 5 Finalizado.

Figura 7. Tablero Kanban del proyecto ClinicaSystem en GitHub Projects.



Matriz de riesgos

#	RIESGO	ACCIÓN DE RESPUESTA
1	Retraso en la migración de repositorios restantes a DbHelper por carga académica de otras materias	Priorizar el refactor de un repositorio a la vez y fijar fechas límite cortas por tarea
2	Fallos de conexión o configuración del entorno SQL Server al probar en otra máquina	Documentar el proceso de configuración del entorno (cadena de conexión, requisitos) en el README
3	Pérdida de datos de prueba durante las pruebas funcionales de login y CRUD	Trabajar sobre una base de datos de pruebas separada de la de producción/entrega final
4	Falta de cobertura de pruebas en casos límite (ej. bloqueo de cuenta, roles no autorizados)	Elaborar un checklist de casos de prueba antes de ejecutar las pruebas funcionales

El uso de campos personalizados (Tipo, Prioridad, Responsable, Fecha estimada) permite ordenar el trabajo según impacto real, evitando que tareas críticas de seguridad como la autenticación (#3, #8) queden relegadas frente a tareas de menor prioridad. Al vincular cada tarjeta con un issue real del repositorio, además, se mantiene trazabilidad directa entre la planificación y el código, lo cual conecta este tablero con la sección de control técnico.

7. Comunicación técnica

Dado que ClinicaSystem fue desarrollado de forma individual, la comunicación técnica del proyecto no giró en torno a un equipo interno, sino a la relación entre el desarrollador y los interesados (stakeholders) del curso.

Interesados identificados

◆ **Facilitador(a) de la asignatura:** principal interesado académico; recibe evidencia del avance mediante las entregas semanales (Semana III a VI) y validará este informe final.

◆ **Usuarios finales simulados (Administrador, Recepcionista, Médico):** representados a través de los requisitos y casos de uso, cuyas necesidades guiaron las decisiones de diseño de interfaz y control de acceso por rol.

◆ **El propio desarrollador,** en su doble rol de analista, diseñador e implementador, responsable de documentar cada decisión técnica para asegurar la trazabilidad del proyecto.

Mecanismos de comunicación empleados

◆ **Entregas semanales estructuradas** (documentos PDF/Word por semana), que funcionaron como reportes formales de avance ante el facilitador, mostrando decisiones tomadas y su justificación técnica.

◆ **Issues de GitHub,** usados para registrar y comunicar necesidades específicas del sistema, permitiendo trazabilidad entre requisito, tarea y código.

◆ **Tablero de GitHub Projects**, como canal de comunicación visual del estado del proyecto: cualquier interesado puede ver en tiempo real qué está finalizado, en progreso o pendiente, sin necesidad de reuniones adicionales.

◆ **Historial de commits** y mensajes de commit descriptivos, que documentan de forma técnica cada cambio realizado (por ejemplo, los refactors de UsuarioRepository, PacienteRepository y MedicoRepository hacia el uso de DbHelper), comunicando decisiones de diseño a cualquiera que revise el repositorio.

En conjunto, este esquema de comunicación combinó reportes formales periódicos (para fines académicos) con herramientas de comunicación asíncrona y trazable propias del flujo de trabajo de ingeniería de software (Issues, commits, tablero Kanban), asegurando que tanto las decisiones como las validaciones del proyecto quedaran documentadas y accesibles.

8. Control técnico y seguimiento

El control técnico de ClinicaSystem se apoya en tres pilares: el repositorio como fuente única de verdad, la documentación del proyecto (README) y el registro de cambios significativos mediante control de versiones.

Repositorio

El código fuente completo se aloja en github.com/Carlosrose07/ClinicaSystem, bajo control de versiones Git, con un flujo de trabajo basado en terminal (git status, git add, git commit -m, git push) que mantiene un historial claro y descriptivo de cada cambio.

Figura 8. Repositorio Carlosrose07/ClinicaSystem en GitHub.

	Assignees	Status	Linked pull requests	Sub-issues progress	Prioridad	Responsable	Tipo	Fecha estimada
1. CRUD de Pacientes #1		Finalizado			Alta	Analista / Desarrollador pr...	Base de datos	Sep 1, 2026
2. CRUD de Doctores y Citas #2		Finalizado			Alta	Responsable de moderado	Base de datos	Sep 3, 2026
3. Autenticación y control de roles #3		En revisión			Alta	Apoyo técnico / Seguridad	Validación	Sep 10, 2026
4. Diseño del modelo Entidad-Relación #4		Finalizado			Media	Responsable de moderado	Modelado	Aug 15, 2026
5. Clase de acceso a datos (DbHelper) #5		En progreso			Media	Desarrollador principal	Base de datos	Sep 20, 2026
6. Documentación técnica inicial #6		En progreso			Baja	Responsable de document...	Documentación	Sep 18, 2026
7. Consulta de agenda diaria del médico #7		En progreso			Media	Apoyo técnico	Interfaz	Sep 30, 2026
8. Bloqueo de cuenta por intentos fallidos de login #8		Finalizado			Alta	Seguridad	Validación	Sep 5, 2026
9. Definición de arquitectura inicial #9		Finalizado			Media		Arquitectura	Sep 12, 2026
10. Pruebas funcionales de login y CRUD #10		En progreso			Alta	QA	Pruebas	Sep 22, 2026

README

El repositorio incluye un README que documenta la instalación y ejecución del sistema, cubriendo: requisitos previos (Visual Studio, .NET Framework, SQL Server), pasos de instalación (clonar repositorio, abrir la solución, restaurar la base de datos vía script SQL, configurar la cadena de conexión, ejecutar), tecnologías utilizadas, estructura del proyecto (carpetas Models, Data, Forms), roles del sistema y credenciales de prueba iniciales. También incluye un diagrama textual de la arquitectura y capturas de pantalla de la interfaz. Esta documentación permite que cualquier tercero clone y ejecute el sistema sin conocimiento previo del proyecto, cumpliendo un estándar mínimo de control técnico.

Figura 9. README del proyecto ClinicaSystem.

Tecnología | Descripción

- C# | Lenguaje de programación principal
- .NET Framework | Plataforma de desarrollo
- Windows Forms | Interfaz gráfica
- SQL Server Express | Base de datos
- ADO.NET | Acceso a datos
- PBKDF2 | Cifrado de contraseñas
- Visual Studio | Entorno de desarrollo
- Git | Control de versiones
- GitHub | Repositorio del proyecto

Estructura del proyecto

```

ClinicaSystem | Models | Paciente.cs | Medico.cs | Usuario.cs | Rol.cs | Cita.cs | HistorialMedico.cs |
Data | ConexionDB.cs | PacienteRepository.cs | MedicoRepository.cs | UsuarioRepository.cs |
CitaRepository.cs | HistorialMedicoRepository.cs | RolRepository.cs | Forms | FrmLogin.cs | FrmMenuPrincipal.cs |
FrmPacientes.cs | FrmMedicos.cs | FrmCitas.cs | FrmHistorialMedico.cs | FrmUsuarios.cs | Program.cs
  
```

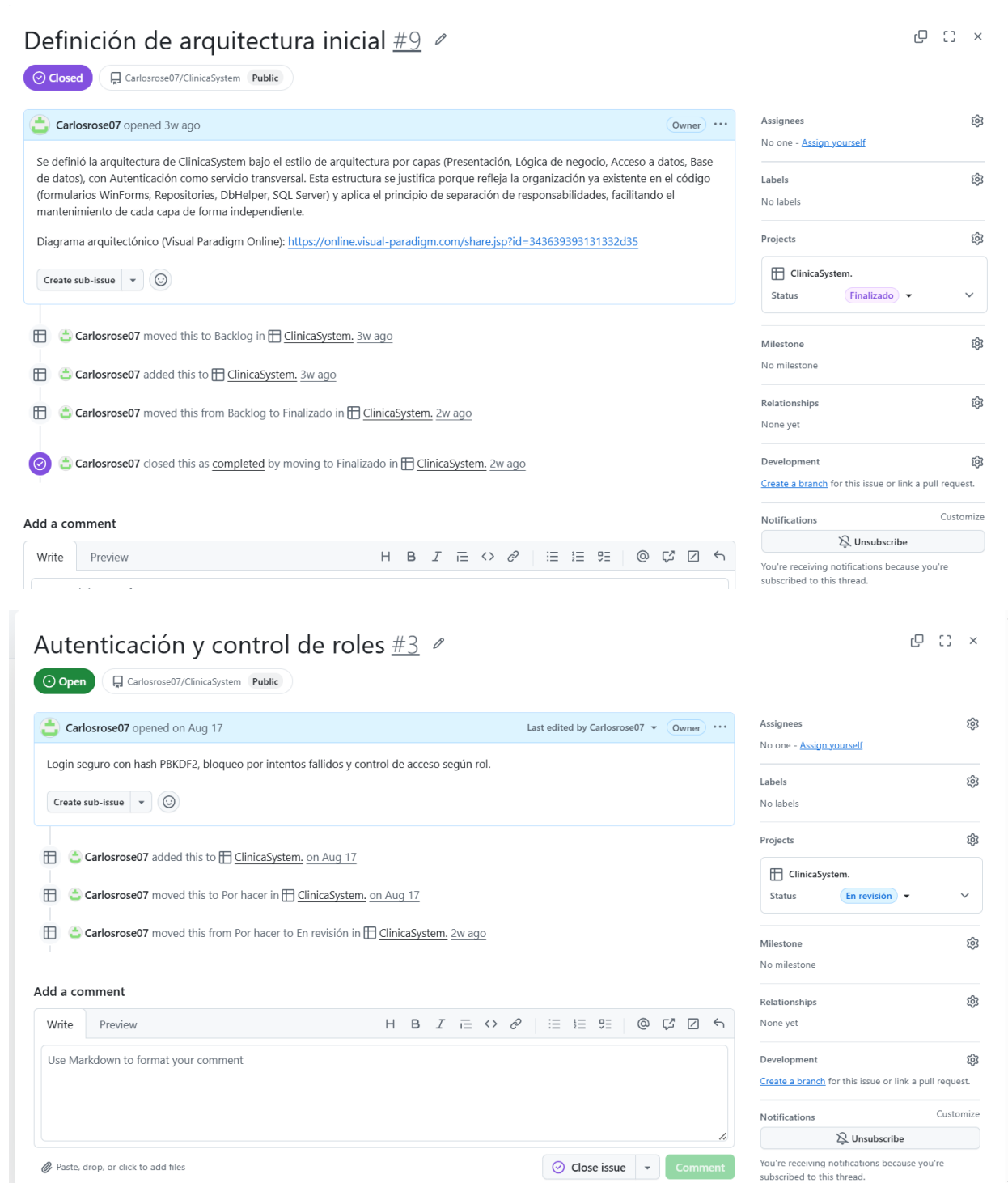
Roles

- Administrador
- Recepcionista
- Médico

Issues

Cada tarea relevante del tablero de GitHub Projects está vinculada a un issue del repositorio (por ejemplo, los issues correspondientes a RF-01 a RF-05 y a la tarjeta "Definición de arquitectura inicial" #9), lo cual conecta directamente la planificación con el código versionado, permitiendo rastrear por qué se hizo un cambio y no solo qué cambió.

Figura 10. Issues del repositorio vinculados a requisitos y tareas del tablero.



Nota técnica de versión

Dado que el README aún no refleja el estado más reciente del sistema, se documenta a continuación como nota técnica de versión, a incorporar en el repositorio como CHANGELOG.md:

- ◆ **Seguridad de contraseñas:** migración del esquema de hash de SHA-256 (documentado originalmente en el README) a PBKDF2 con salt único por usuario, con migración transparente de credenciales existentes al nuevo esquema.
- ◆ **Refactor de acceso a datos:** introducción de la clase centralizada DbHelper (con EjecutarConsulta<T>, EjecutarConsultaUnica<T>, EjecutarNonQuery, EjecutarEscalar<T>) y de DatosException para el manejo unificado de errores SQL. UsuarioRepository fue el primero en migrar a este patrón (01/08/2026), seguido de PacienteRepository y MedicoRepository (07/09/2026), manteniendo la API pública sin cambios.
- ◆ **Pendiente para próxima versión:** completar la migración de CitaRepository y HistorialMedicoRepository a DbHelper, e implementar paginación en listados extensos.

Esta nota evidencia que el sistema no es una entrega estática, sino que ha evolucionado durante el curso a partir de decisiones técnicas documentadas y trazables mediante commits e issues.

Seguridad: consultas parametrizadas

El acceso a datos centralizado en DbHelper aplica consultas parametrizadas en todas las operaciones SQL, evitando la construcción de sentencias por concatenación de texto. El siguiente ejemplo ilustra el patrón aplicado frente a la práctica insegura que reemplaza (adapta las líneas con el código real de tu repositorio antes de entregar):

```
//codigofuente
// ✓ SEGURO – consulta parametrizada (patrón aplicado en DbHelper)
var query = "SELECT * FROM Pacientes WHERE Cedula = @Cedula";
var parametros = new[] { new SqlParameter("@Cedula", cedula) };
var resultado = dbHelper.EjecutarConsulta<Paciente>(query, parametros);

// X INSEGURO – concatenación directa (vulnerable a inyección SQL)
// var query = "SELECT * FROM Pacientes WHERE Cedula = '" + cedula + "'";
```

Buenas prácticas implementadas

PRÁCTICA	IMPLEMENTACIÓN EN EL PROYECTO
Separación de responsabilidades	Arquitectura en capas: Presentación, Lógica de negocio, Acceso a datos y Base de datos.
Encapsulamiento	Repositorios especializados por entidad (PacienteRepository, MedicoRepository, etc.) exponen solo los métodos necesarios.
Gestión de recursos (using)	Conexiones y comandos SQL liberados mediante bloques using dentro de DbHelper.
Prevención de inyección SQL	Consultas parametrizadas con SqlParameter en toda operación de acceso a datos.
Manejo de excepciones	Errores SQL traducidos a DatosException con mensajes en español, sin exponer detalles internos (RNF-05).
Código DRY	Lógica de ejecución de consultas centralizada en DbHelper en lugar de repetirse en cada repositorio.
Tipos nullable	Campos opcionales de la base de datos mapeados con tipos nullable de C# (por ejemplo, fechas o teléfonos no obligatorios).
Nomenclatura clara	Convención consistente de nombres (XxxRepository, FrmXxx) en todo el proyecto.

9. Relación con la interfaz del sistema

El diseño de la interfaz de ClinicaSystem no es independiente de las decisiones de análisis, modelado y arquitectura documentadas en las secciones anteriores; por el contrario, cada pantalla materializa directamente un requisito o una decisión técnica específica.

Pantalla de inicio de sesión (FrmLogin)

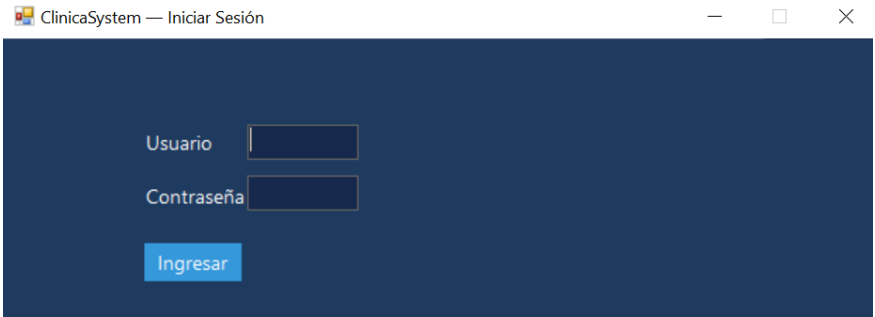


Figura 11. Pantalla de inicio de sesión (FrmLogin).

Esta pantalla implementa directamente RF-03 (autenticación mediante usuario y contraseña) y RNF-01/RNF-02 (contraseñas con hash PBKDF2 y salt, bloqueo tras 5 intentos fallidos). El diseño minimalista —solo dos campos y un botón "Ingresar"— refleja la prioridad del sistema de resolver rápidamente el punto de entrada obligatorio para cualquier rol, sin distraer con elementos adicionales antes de validar identidad. Es también el punto donde se activa el servicio transversal de Autenticación descrito en el diseño arquitectónico: antes de mostrar cualquier otra pantalla, el sistema valida credenciales contra el hash almacenado y, según el rol resultante, determina qué opciones de menú se mostrarán a continuación (RF-04).

Pantalla de Gestión de Pacientes (FrmPacientes)

ID	Nombre	Cédula	Nacimiento	Teléfono
2	Jessica Polanco Reyes	939-0500001-0	22/06/2026	809-555-1004
1	Ramon Rosario Reyes	056-1234799-0	14/07/1999	802-3455-63

Figura 12. Pantalla de Gestión de Pacientes (FrmPacientes).

Esta pantalla implementa directamente RF-01 (registrar, editar, eliminar y listar pacientes con validación de campos obligatorios). Se observa el patrón de interfaz maestro-detalle: una tabla a la izquierda (ID, Nombre, Cédula, Nacimiento, Teléfono) que lista los pacientes existentes —correspondiente a la operación de lectura del CRUD—, y un panel de formulario a la derecha (Nombre completo, Cédula, Fecha de nacimiento, Teléfono, Dirección, Tipo de sangre) para crear o editar un registro. Los botones Nuevo, Guardar, Eliminar y Cancelar, diferenciados por color (azul, verde, rojo, gris), corresponden de forma directa a las cuatro operaciones CRUD exigidas por RF-01, y su codificación visual reduce el riesgo de que el usuario confunda una acción destructiva (Eliminar) con una constructiva (Guardar), lo cual conecta con el criterio de usabilidad de RNF-03. El campo "Buscar" en la parte superior de la tabla, además, facilita la localización de un paciente entre múltiples registros, relevante para el criterio de desempeño de RNF-04.

En ambas pantallas se mantiene el tema oscuro (#1E3A5F / #16284B) establecido como estándar visual del sistema, lo que demuestra consistencia de diseño a través de los distintos módulos, más allá de la funcionalidad puntual de cada formulario.

Estas evidencias muestran que la interfaz no fue diseñada de forma aislada, sino como la capa de Presentación descrita en la arquitectura en capas: cada elemento visual tiene una correspondencia directa con un requisito validado, una regla de negocio o una decisión de seguridad documentada en las secciones anteriores del informe.

CONCLUSIÓN TÉCNICA

El desarrollo de ClinicaSystem, documentado a lo largo de este informe, demuestra la aplicación coherente de los fundamentos de la ingeniería de software sobre un proyecto real y funcional, más allá de un ejercicio puramente teórico. Desde el levantamiento de requisitos funcionales y no funcionales con criterios de aceptación verificables, pasando por el modelado UML de los flujos críticos del sistema y el diseño de una arquitectura en capas justificada técnicamente, hasta la gestión ágil del avance mediante un tablero Kanban en GitHub Projects, cada etapa del curso contribuyó a un producto de software con una base técnica sólida y trazable.

Entre los logros técnicos más relevantes del proyecto destacan:

- ◆ Un conjunto completo de requisitos funcionales y no funcionales, cada uno con un criterio de aceptación verificable y reflejado en la implementación real del sistema.
- ◆ Cinco diagramas UML coherentes entre sí, que representan el sistema desde perspectivas funcionales, estáticas y de comportamiento complementarias.
- ◆ Una arquitectura en capas justificada técnicamente, alineada con la organización real del código fuente y con el principio de separación de responsabilidades.
- ◆ Un mecanismo de autenticación con almacenamiento seguro de credenciales (PBKDF2 con salt) y bloqueo de cuenta ante intentos fallidos.
- ◆ Un tablero Kanban en GitHub Projects con trazabilidad completa entre tareas, issues y commits del repositorio.
- ◆ Documentación técnica de control (README, nota de versión) que permite a un tercero comprender y ejecutar el sistema sin conocimiento previo del proyecto.

Un hallazgo particularmente relevante de este proceso de integración fue detectar que la documentación del repositorio (README) había quedado desactualizada respecto a la implementación real de seguridad de SHA-256 a PBKDF2 con salt, lo cual evidenció en la práctica la importancia del control técnico y de mantener una nota de versión actualizada, un principio que suele quedar solo en la teoría en proyectos académicos pero que aquí se verificó de forma concreta.

En conjunto, el proyecto reafirma que la calidad de un sistema de software no depende únicamente del código funcional, sino de la disciplina con que se documentan, comunican y trazan las decisiones técnicas que lo sustentan: requisitos, modelos, arquitectura, gestión y control de versiones forman un ciclo interdependiente, y ClinicaSystem con sus CRUDs completos, su control de acceso por rol y su capa de seguridad reforzada es evidencia de que ese ciclo se aplicó de forma consistente durante el curso de Ingeniería de Software I. Esta base técnica documentada deja el proyecto en condiciones de crecer de forma ordenada: completar los repositorios pendientes, incorporar pruebas automatizadas y mantener la trazabilidad entre planificación y código en futuras iteraciones del sistema.

REFERENCIAS BIBLIOGRÁFICAS

- Anderson, D. J. (2010). Kanban: Successful evolutionary change for your technology business. Blue Hole Press.
- Bass, L., Clements, P., & Kazman, R. (2013). Software architecture in practice (3rd ed.). Addison-Wesley.
- Fowler, M. (2003). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley Professional.
- Kaliski, B. (2000). PKCS #5: Password-based cryptography specification version 2.0 (RFC 2898). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc2898>
- Sommerville, I. (2016). Software engineering (10th ed.). Pearson.